Time series analysis

AB–204 Arctic population ecology, October 27–30, 2026

J.-Prof. Dr. Timo Adam

Organisational issues

- **Lecturer**: Timo Adam, Bielefeld University, Germany (timo.adam@uni-bielefeld.de).
- **Lectures** in the morning: the main ideas of time series analysis.
- **Practicals** in the afternoon: you'll analyse your own data in `R` — with our help.
- **Prerequisites**: none — we'll start (more or less) from scratch.
- **Outcome**: a **poster** presenting the analysis of your own data (prepared after the course).
- **Material**: slides, exercise sheets, and data sets are on the course website.
- **Introductory round**: who are you, and what's your background?

Organisational issues

- **Questions** can be asked at any time — also **anonymously** via Mentimeter:

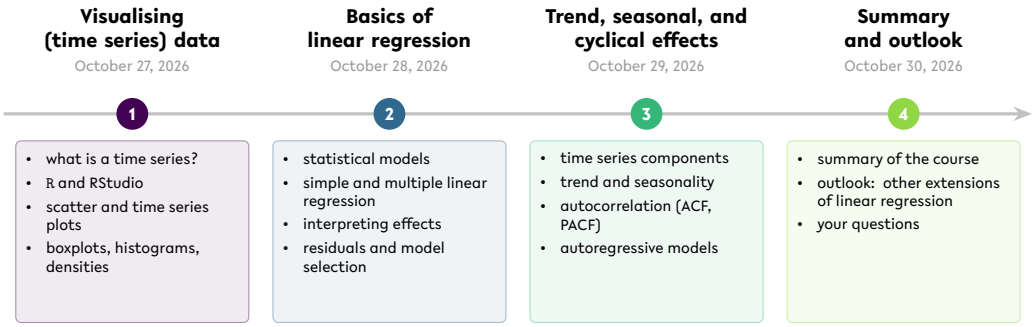
  www.menti.com/al94rhvnrre9

- I will check the questions regularly and answer them in the lecture.
- No question is too basic — if something is unclear to you, it is probably unclear to others as well.



Scan to ask a question

Course overview



Aims of the course

After this course, you will be able to:

- **explore** and **visualise** ecological (time series) data in R;
- **fit** and **interpret** regression models;
- **model** trend, seasonal, and cyclical patterns;
- **check** whether a model is adequate;
- **present** your analysis on a poster.

More generally: translate an ecological question into a statistical model, and critically assess what the model can (and cannot) tell us.

Lecture 1: Visualising (time series) data

1.1 Introduction — what is a time series?

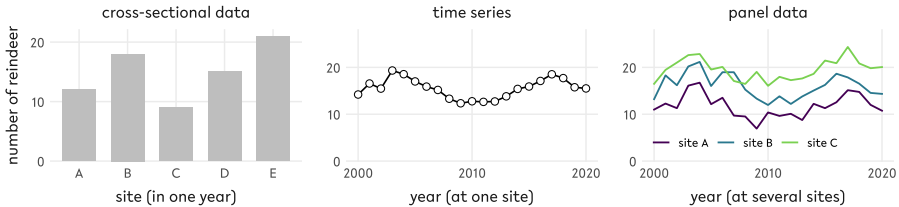
1.2 Working with data in R

1.3 Scatter plots and time series plots

1.4 Visualising distributions

1.1 Introduction — types of data

- **Cross-sectional data**: several units (e.g., sites) at **one** point in time.
- **Time series**: **one** unit at several points in time.
- **Panel data**: several units at several points in time.



(simulated data)

1.1 Introduction — quiz

Which type of data are the following?

1. The number of reindeer counted at 20 sites on Svalbard in 2025.
2. The number of reindeer counted in Adventdalen every year from 1979 to 2025.
3. The number of reindeer counted at 20 sites every year from 1979 to 2025.
4. The GPS positions of 30 muskoxen, recorded every hour for one year.
5. The behaviour (resting, foraging, or travelling) of one seal, recorded every minute for one day.

In this course, we'll focus on **time series**.

1.1 Introduction — what is a time series?

Definition:

A **time series** is a sequence of observations $y_1, y_2, \ldots, y_T$ collected over time, where

- $t = 1, \ldots, T$ is the time index (in chronological order);
- T is the number of observations.

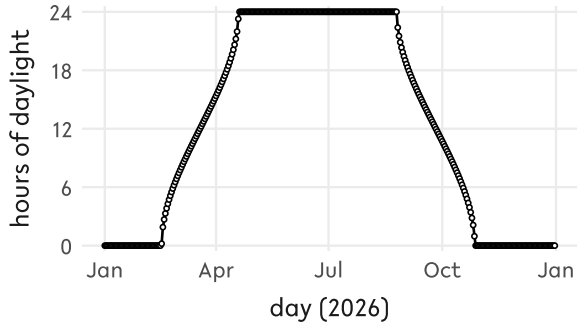
Some examples:

- temperature$_t$, $t = 1, \ldots, 365$ (daily air temperature in Longyearbyen);
- polar bear sightings$_t$, $t = 1, \ldots, 12$ (monthly number of sightings);
- sea ice extent$_t$, $t = 1979, \ldots, 2025$ (annual minimum Arctic sea ice extent).

1.1 Example 1 — daylight in Longyearbyen



Sunset near Longyearbyen



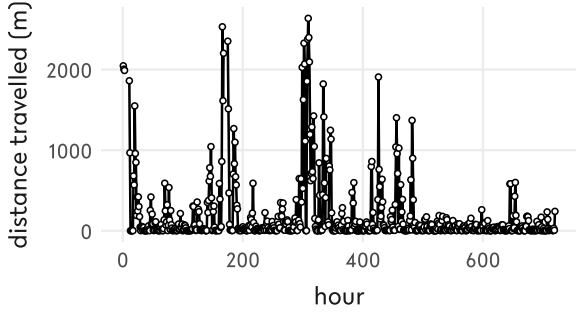
Hours of daylight in Longyearbyen (78° N)[1] — one observation per **day**.

[1] Calculated from the position of the sun, using the NOAA solar equations.

1.1 Example 2 — movement of a muskox



muskox (*Ovibos moschatus*)



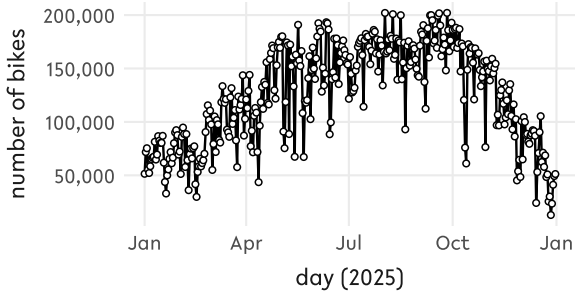
Hourly distances travelled by a muskox in north-east Greenland[2] — one observation per **hour**.

[2]Data from Beumer, L., et al. (2020): An application of upscaled optimal foraging theory using hidden Markov modelling: year-round behavioural variation in a large arctic herbivore. *Movement Ecology*, 8, 25, doi:10.1186/s40462-020-00213-x.

1.1 Example 3 — bike sharing in New York City



Citi Bike station in New York City



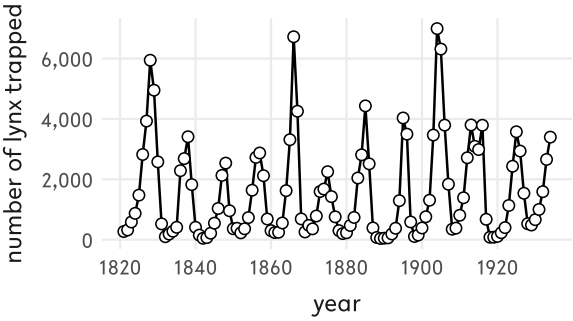
Daily number of rented bikes in New York City in 2025[3] — one observation per **day**.

[3] Data from https://citibikenyc.com/system-data.

1.1 Example 4 — Canadian lynx



Canada lynx (*Lynx canadensis*)



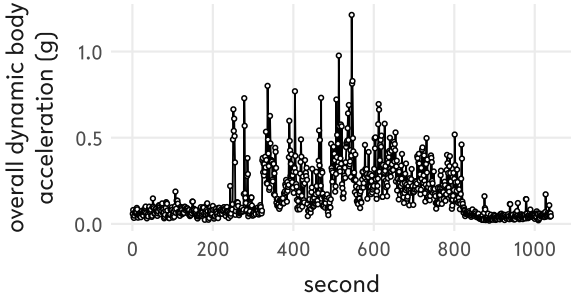
Number of lynx trapped in north-west Canada, 1821–1934[4] — one observation per **year**.

[4]Data from Campbell, M. and Walker, A. (1977): A survey of statistical work on the Mackenzie River series of annual Canadian lynx trappings for the years 1821–1934 and a new analysis. *Journal of the Royal Statistical Society, Series A*, 140, 411–431, doi:10.2307/2345277; available as `lynx` in R.

1.1 Example 5 — activity of an oceanic whitetip shark



oceanic whitetip shark (*Carcharhinus longimanus*)



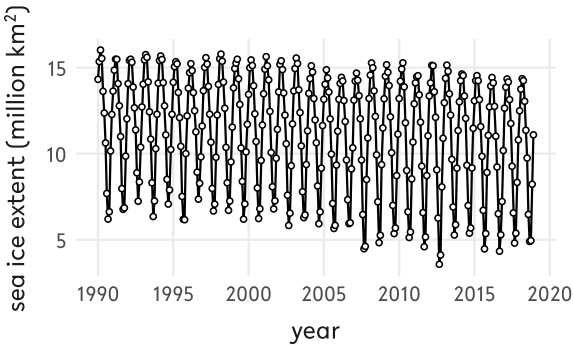
Body acceleration (a measure of activity) of an oceanic whitetip shark[5] — one observation per **second**.

[5] Data from Papastamatiou, Y., et al. (2022): Individual differences in diel and vertical activity patterns in a large pelagic predator, the oceanic whitetip shark. *Marine Biology*, 169, 137, doi:10.1007/s00227-022-04119-7.

1.1 Example 6 — Arctic sea ice extent



sea ice

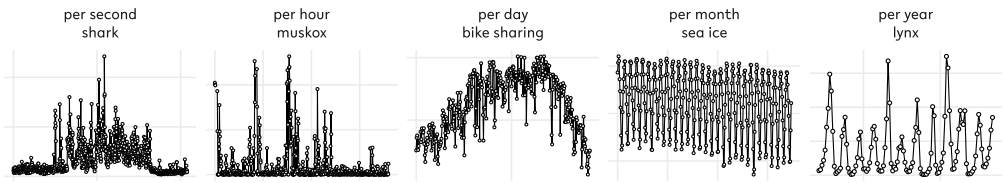


Monthly Arctic sea ice extent, 1990–2018[6] — one observation per **month**.

[6] Data from Cavalieri, D., Parkinson, C., Gloersen, P., and Zwally, H. (1996): Sea ice concentrations from Nimbus-7 SMMR and DMSP SSM/I-SSMIS passive microwave data, version 1. NASA National Snow and Ice Data Center, doi:10.5067/8GQ8LZQVL0VL; daily extents from https://www.kaggle.com/datasets/nsidcorg/daily-sea-ice-extent-data; in the course data set `sea_ice.csv`.

1.1 Introduction — from seconds to years

The examples cover a wide range of **temporal resolutions**:



- The resolution determines which patterns we can see: daily cycles need at least hourly data, seasonal cycles at least monthly data.
- Choose the resolution to match the question (and aggregate if necessary, e.g., from hours to days).

1.1 Introduction — characteristics of time series

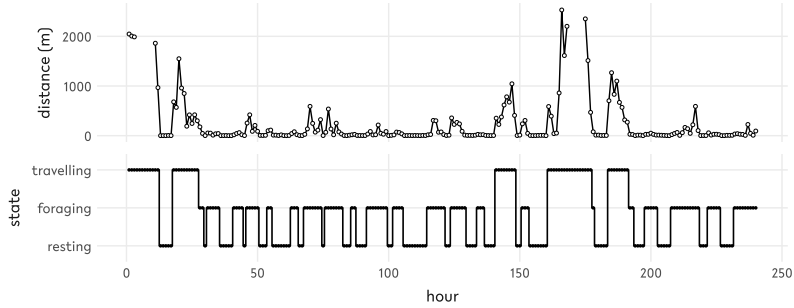
Time series can be:

- **equally** or **unequally spaced** in time;
- **short** or **long** (one day vs. 100 years);
- of different **resolutions** (per second vs. per year);
- of different **scales of measurement**:
  - **nominal**: categories (e.g., resting, foraging, travelling);
  - **ordinal**: ordered categories (e.g., no, partial, full snow cover);
  - **metric**: numbers (e.g., number of lynx, temperature).

We'll focus on metric time series that are equally spaced in time.

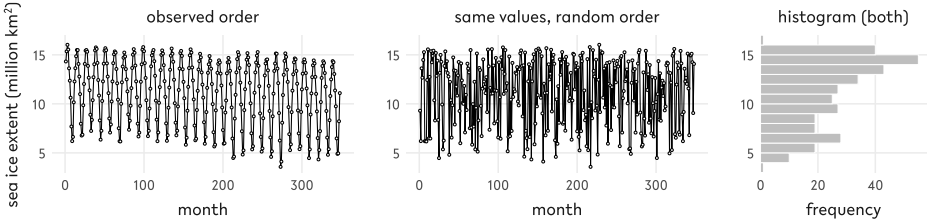
1.1 Introduction — a categorical time series

The muskox from Example 2: the **distance** travelled per hour (metric) and the **behavioural state** (nominal) in the first 10 days:



The states were not observed, but inferred from the distances using a **hidden Markov model** (not covered in this course).

The sea ice data from Example 6 — in the observed order, and the **same values** in a random order:



- Same values, same histogram — but only the observed series shows the annual cycle.
- The **order** matters: observations close in time tend to be **similar** (Lecture 3).

1.1 Introduction — aims of time series analysis

Possible **aims**:

- **Describing** the dynamics (how does a population change over time?);
- **Explaining** the dynamics (why — temperature, sea ice, predators?);
- **Detecting changes** (did the dynamics change after a harsh winter?);
- **Inferring hidden states** (when is an animal resting, foraging, or travelling?);
- **Forecasting** (how will the population develop in the next years?).

Whatever the aim: first **look at the data** — which is what we'll do today.

Lecture 1: Visualising (time series) data

1.1 Introduction — what is a time series?

1.2 Working with data in R

1.3 Scatter plots and time series plots

1.4 Visualising distributions

- We'll use R and **RStudio**[7].
- R does the calculations (the engine); RStudio is a convenient interface to it (the dashboard).
- Please make sure that both run on your laptop before the first practical.



[7] Both are free: first install R from https://cran.r-project.org, then RStudio from https://posit.co/download/rstudio-desktop.

1.2 Working with data in R — the four panes of RStudio



1 write your code; **2** results; **3** your objects (e.g., data); **4** plots and help pages.

1.2 Working with data in R — R as a calculator

R can be used as a calculator:

```
2 + 3 # 5
log(10) # natural logarithm: 2.302585
```

Results can be stored in **objects** using the **assignment operator** <-:

```
x <- 5 # store the number 5 in an object called x
x * 2 # 10
```

Everything right of # is a **comment**: it won't be executed — use comments to explain your code.

1.2 Working with data in R — vectors and functions

Several values can be combined into a **vector** using `c()`:

```
counts <- c(12, 18, 9, 15, 21) # reindeer counted at five sites
```

Functions do something with their **arguments**:

```
mean(counts) # 15
max(counts) # 21
round(3.14159, digits = 2) # 3.14
```

Help on any function — the examples at the bottom of the help page are particularly useful:

```
?mean
```

1.2 Working with data in R — scripts and packages

Scripts: write your code in a script (File → New File → R Script):

- run a line with Ctrl + Enter (Windows) or Cmd + Enter (Mac);
- save the script, so that your analysis can be repeated at any time.

Packages extend R by further functions:

```
install.packages("viridisLite") # install a package (only once)
library(viridisLite) # load it (in every session)
```

1.2 Working with data in R — common pitfalls

When R returns an error, check the following first:

- R is **case-sensitive**: `Count` is not the same as `count`.
- Every opening bracket needs a **closing bracket**.
- The decimal separator is a **point**: `2.5`, not `2,5`.
- Is the data file in the **working directory**?
- A + in the console means that R is waiting for the rest of a command — press Esc.

Copying the error message into a search engine (or asking us) usually helps!

1.2 Working with data in R — the bike sharing data

Daily number of rented bikes in New York City, January 2016 – September 2026, with weather data from Central Park[8]:

variable	description	unit
`date`	date	year-month-day
`year`, `month`, `week`	year, month, week	—
`day_of_week`	day of the week	0 = Monday, ..., 6 = Sunday
`day_of_year`	day of the year	1, ..., 366
`windspeed`	average wind speed	mph
`precipitation`, `snowfall`	precipitation, snowfall	inches
`min_temperature`, `max_temperature`	min. and max. temperature	°F
`count`	number of rented bikes	—

Table: Variables in `bike_sharing.csv` (one row per day, 3,926 days).

($°C = (°F - 32) \cdot 5/9$; 1 mph $= 1.6$ km/h; 1 inch $= 25.4$ mm.)

[8] Rentals from https://citibikenyc.com/system-data; weather from NOAA's Global Historical Climatology Network (station New York Central Park), doi:10.7289/V5D21VHZ.

1.2 Working with data in R — importing data

Import a data file:

```
setwd("~/Desktop/AB-204") # the folder with the data
bikes <- read.csv("bike_sharing.csv") # assign the data to the object bikes
```

Tip: create an RStudio **project** (File → New Project) in the folder with your data — then you don't need `setwd()`.

1.2 Working with data in R — a first look

Get an **overview** of the data:

```
dim(bikes) # number of rows and columns: 3926 12
str(bikes) # type of each variable and its first values
summary(bikes) # summary statistics for each variable
head(bikes) # first 6 rows
tail(bikes) # last 6 rows
```

The first rows of some of the variables — one **row** per day, one **column** per variable:

```
        date windspeed min_temperature count
1 2016-01-01      7.61              34 11009
2 2016-01-02      6.71              32 14587
3 2016-01-03      7.83              35 15499
```

1.2 Working with data in R — accessing elements

Elements of the data, using `bikes[row, column]`:

```
bikes[1, ] # first row
bikes$count # the column called count
```

Elements of a variable, using `variable[position]`:

```
bikes$windspeed[3] # third element: 7.83
bikes$min_temperature[1:4] # first 4 elements: 34 32 35 14
```

(`1:4` is short for `c(1, 2, 3, 4)`.)

1.2 Working with data in R — functions and subsets

Apply **functions** to variables:

```
mean(bikes$count, na.rm = TRUE) # mean number of bikes: 77928.09
max(bikes$windspeed, na.rm = TRUE) # largest wind speed: 18.34
sum(bikes$precipitation == 0) # days without rain: 2518
which(bikes$count > 210000) # rows with more than 210,000 bikes: 3907 3914
```

Create a **subset** of the data:

```
bikes_2025 <- subset(bikes, year == 2025) # data from 2025
```

(== means “is equal to?”, != means “is not equal to?”.)

1.2 Working with data in R — missing values

Missing values are coded as `NA` (“not available”):

```
sum(is.na(bikes$windspeed)) # number of missing wind speeds: 223
which(is.na(bikes$windspeed)) # their positions: 134 135 136 137 ...
mean(bikes$windspeed) # NA
mean(bikes$windspeed, na.rm = TRUE) # 5.03
```

Many functions return `NA` unless you remove the missing values with `na.rm = TRUE`.

1.2 Working with data in R — filling gaps

A **single** missing value can be filled by **linear interpolation** — the mean of its two neighbours:

```
bikes$windspeed[138:140] # 3.80 NA 1.34
bikes$windspeed[139] <- mean(bikes$windspeed[c(138, 140)]) # 2.57
```

Only do this for isolated gaps — not for long ones (e.g., 149 missing days in a row in winter 2018/19).

Always report how you handled missing values.

1.2 Working with data in R — new variables

Create **new variables** from existing ones:

```
bikes$average_temperature <- (bikes$min_temperature + bikes$max_temperature) / 2
bikes$rainy <- as.numeric(bikes$precipitation > 0) # 1 = rainy, 0 = dry
```

Store the date as a **date** (needed for time series plots):

```
bikes$date <- as.Date(bikes$date)
```

(Never overwrite your raw data — do all changes in your script.)

Lecture 1: Visualising (time series) data

1.1 Introduction — what is a time series?

1.2 Working with data in R

1.3 Scatter plots and time series plots

1.4 Visualising distributions

1.3 Colours

Use **black** and **grey** by default — and colours only to **distinguish categories** (e.g., years, species, rainy and dry days). Here, colours from the **viridis** palette[9] — colour-blind friendly, and readable in black and white:

viridis(7, end = 0.9)



One colour per category:

```
library(viridisLite) # viridis colours (install.packages("viridisLite") once)
pal <- viridis(7, end = 0.9) # 7 colours, e.g., one per year
```

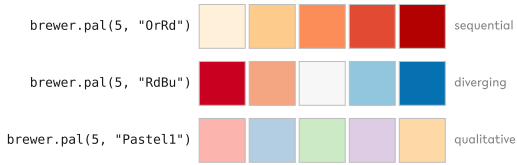
(`adjustcolor("black", alpha.f = 0.3)` makes a colour transparent — useful for overlapping points; `alpha("black", 0.3)` from the `scales` package does the same.)

[9] R package `viridisLite`, https://CRAN.R-project.org/package=viridisLite.

1.3 Colours — finding colour schemes

A good source for colour schemes is **ColorBrewer**: colorbrewer2.org

- **sequential**: ordered values from low to high (e.g., temperature);
- **diverging**: deviations in both directions from a midpoint (e.g., anomalies);
- **qualitative**: categories without an order (e.g., species).



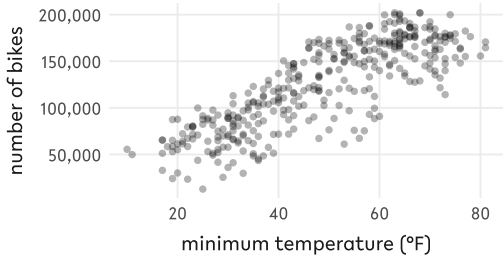
The website lets you filter for colour-blind safe and print-friendly schemes. In R:

```
library(RColorBrewer) # install.packages("RColorBrewer") once
pal <- brewer.pal(5, "OrRd") # 5 colours from the scheme OrRd
```

1.3 Scatter plots

Scatter plots show the **relationship** between two variables — here for 2025:

```
plot(x = bikes_2025$min_temperature, y = bikes_2025$count, pch = 20,
     col = adjustcolor("black", alpha.f = 0.3),
     xlab = "minimum temperature (F)", ylab = "number of bikes")
```

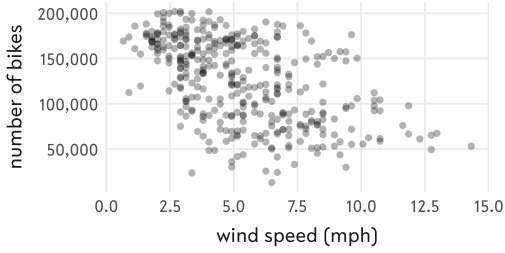


More bikes are rented on warm days.

1.3 Scatter plots

The same for wind speed:

```
plot(x = bikes_2025$windspeed, y = bikes_2025$count, pch = 20,
     col = adjustcolor("black", alpha.f = 0.3),
     xlab = "wind speed (mph)", ylab = "number of bikes")
```

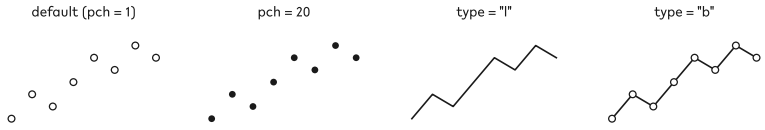


Fewer bikes are rented on windy days.

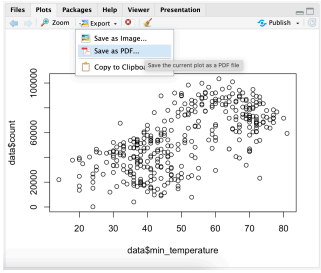
1.3 Scatter plots — customising plots

Most plotting functions share the same **arguments**:

argument	meaning	example
`xlab`, `ylab`	axis labels (with units!)	`xlab = "wind speed (mph)"`
`main`	title	`main = ""` (no title)
`xlim`, `ylim`	range of the axes	`ylim = c(0, 100000)`
`col`	colour	`col = "grey"`
`cex`	size of symbols	`cex = 1.5`
`pch`	plotting symbol	see below
`type`	points, lines, or both	see below



1.3 Scatter plots — saving plots



Option 1 — in RStudio, click on Export → Save as Image/PDF.

Option 2 — recommended, as it is reproducible:

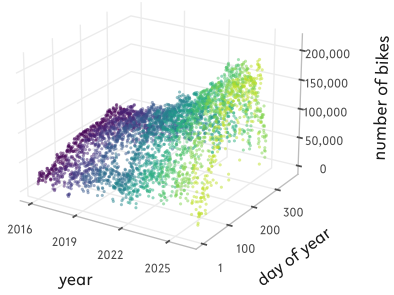
```
pdf("my_plot.pdf", width = 7, height = 5)
plot(x = bikes$min_temperature, y = bikes$count)
dev.off() # close the file
```

(For your poster: `png("my_plot.png", width = 2000, height = 1400, res = 300)`.)

1.3 Scatter plots — three dimensions

Three variables at once, using the `simply3d` package[10]:

```
library(simply3d)
d <- subset(bikes, !is.na(count)) # days with a count
simply_scatter(x = d$year, y = d$day_of_year, z = d$count,
               color = viridis(11, end = 0.9)[d$year - 2015]) # one colour per year
```



In R, you can rotate and zoom in — try it out!

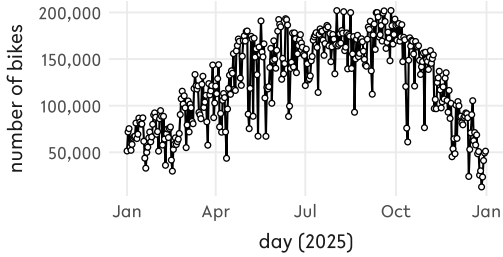
More bikes in summer, and more bikes from year to year.

[10] https://github.com/janoleko/simply3d; install it with `remotes::install_github("janoleko/simply3d")`.

1.3 Time series plots

A **time series plot** has **time** on the x-axis — `type = "b"` draws **both** points and lines (in black, the default colour):

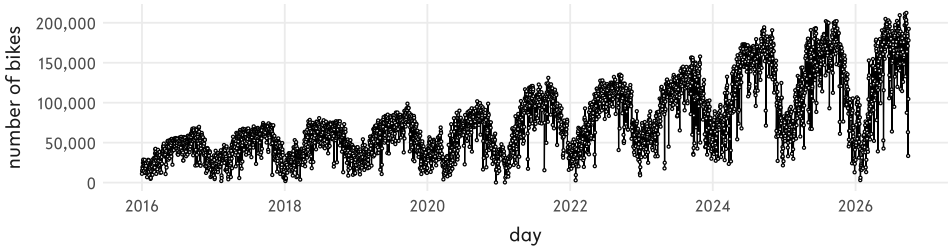
```
plot(x = bikes_2025$date, y = bikes_2025$count, type = "b", cex = 0.6,
     xlab = "day (2025)", ylab = "number of bikes")
```



1.3 Time series plots

All years — for long time series, smaller symbols (`cex`) keep the plot readable:

```
plot(x = bikes$date, y = bikes$count, type = "b", cex = 0.3,
     xlab = "day", ylab = "number of bikes")
```



What patterns can you identify? (More in Lecture 3.)

Lecture 1: Visualising (time series) data

1.1 Introduction — what is a time series?

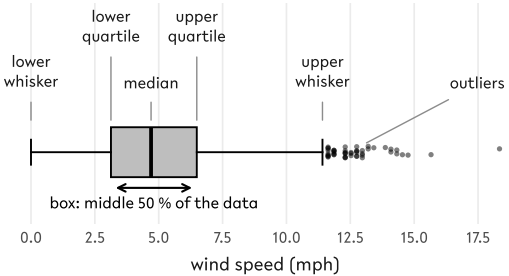
1.2 Working with data in R

1.3 Scatter plots and time series plots

1.4 Visualising distributions

1.4 Visualising distributions — boxplots

Boxplots summarise the **distribution** of a variable — here the daily wind speed:

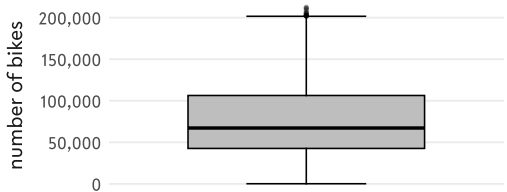


- **median**: half of the values are smaller, half are larger;
- **box**: the middle 50 % of the values;
- **whiskers**: up to 1.5 box lengths;
- **outliers**: unusual values beyond.

1.4 Visualising distributions — boxplots

A boxplot of the daily number of rented bikes:

```
boxplot(x = bikes$count, ylab = "number of bikes",
        col = "grey")
```

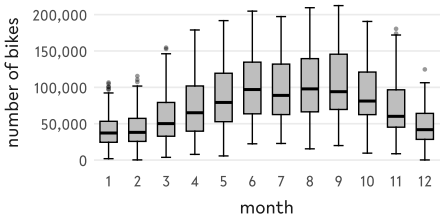


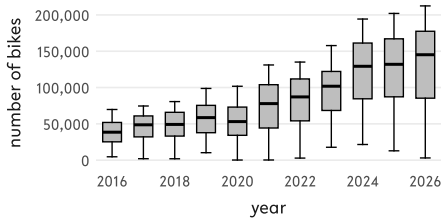
On half of the days, between about 34,000 and 96,000 bikes were rented.

1.4 Visualising distributions — boxplots by group

`count ~ month` means "`count` split by `month`":

```
boxplot(count ~ month, data = bikes, xlab = "month", ylab = "number of bikes",
        col = "grey", outpch = 20)
```

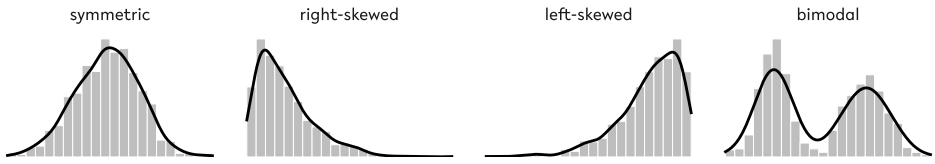




More bikes in summer (seasonal pattern), and in later years (trend; same code with `count ~ year`).

1.4 Visualising distributions — histograms

Histograms show the **shape** of a distribution — typical shapes:



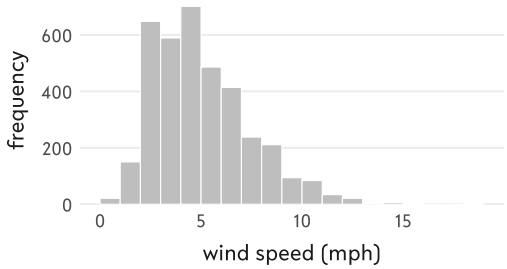
(simulated data)

- **Skewed**: many small, few large values (e.g., counts, distances).
- **Bimodal**: two peaks (often a hint at two groups, e.g., males and females).

1.4 Visualising distributions — histograms

A histogram of the daily wind speed:

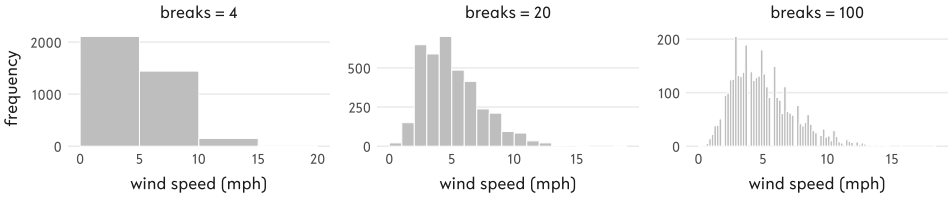
```
hist(x = bikes$windspeed, breaks = 20, main = "", col = main, border = "white",
     xlab = "wind speed (mph)", ylab = "frequency")
```



The wind speed is **right-skewed**.

1.4 Visualising distributions — how many bins?

`breaks` controls the (approximate) number of bins:

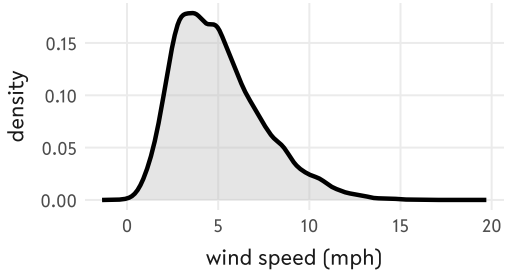


Too few bins hide the shape, too many make the histogram noisy.

1.4 Visualising distributions — kernel density estimates

Kernel density estimates are a **smooth** version of a histogram:

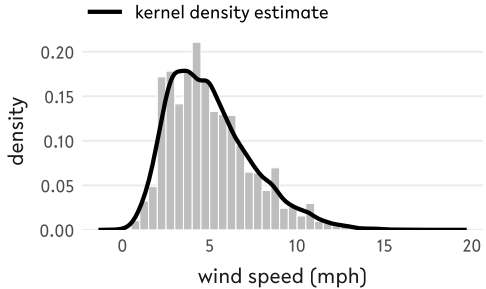
```
dens <- density(x = bikes$windspeed, na.rm = TRUE)
plot(dens, lwd = 2, xlab = "wind speed (mph)", ylab = "density", main = "")
polygon(dens, col = adjustcolor("grey", alpha.f = 0.5), border = NA) # shade the area
```



1.4 Visualising distributions — kernel density estimates

Histogram and kernel density estimate together:

```
hist(x = bikes$windspeed, breaks = 40, probability = TRUE, main = "",
     col = "grey", border = "white",
     xlab = "wind speed (mph)", ylab = "density")
lines(dens, lwd = 2) # add to the histogram
```

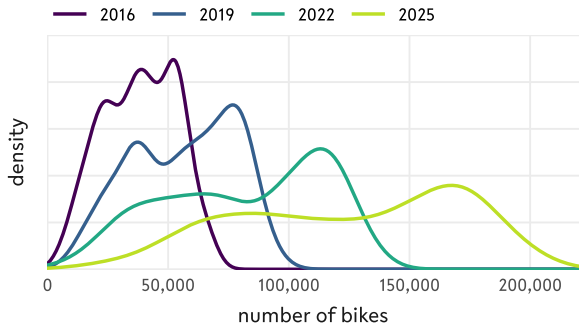


1.4 Visualising distributions — comparing years

A **for loop** repeats the same commands for each of four years:

```
years <- c(2016, 2019, 2022, 2025)
pal <- viridis(4, end = 0.9) # 4 colours, one per year
plot(x = 0, type = "n", xlim = c(0, 220000), ylim = c(0, 0.000025),
     xlab = "number of bikes", ylab = "density", main = "") # empty plot
for(i in 1:4) { # for i = 1, 2, 3, 4
  dens <- density(x = subset(bikes, year == years[i])$count, na.rm = TRUE)
  lines(dens, col = pal[i], lwd = 2) # add the density of year i
}
legend("topright", legend = years, col = pal, lwd = 2, bty = "n")
```

1.4 Visualising distributions — comparing years



The distribution shifts to the right over the years.

Some years have **two** peaks — why? (Hint: Lecture 3.)

Lecture 1 — summary

plot	shows	in R
scatter plot	relationship between two variables	`plot(x, y)`
3D scatter plot	relationship between three variables	`simply_scatter(x, y, z)`
time series plot	one variable over time	`plot(date, y, type = "b")`
boxplot	summary of a distribution	`boxplot(y ~ group)`
histogram	shape of a distribution	`hist(y)`
kernel density estimate	smooth histogram	`plot(density(y))`

Table: Overview of today's plots.

Always **plot your data first** — trends, seasonality, outliers, and gaps are obvious in a plot, but invisible in a table.

Lecture 1 — summary questions

1. What is a time series, and how does it differ from cross-sectional and panel data? Give an ecological example of each.
2. Why does the order of the observations matter for time series — what would you lose by shuffling them?
3. How do you import a data set into `R`, and which functions give you an overview of it?
4. How are missing values coded in `R`? When is linear interpolation a sensible way to fill them — and when not?
5. Which plot would you use to show (a) the relationship between two variables, (b) a variable over time, (c) the distribution of a variable?
6. What do the box, the line inside it, and the whiskers of a boxplot show? What does a histogram show that a boxplot doesn't?

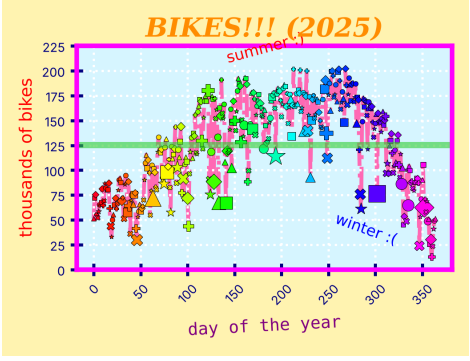
Homework — the crazy plot

Create the **craziest plot** of a data set of your own:

- show your data and tell a story — what's interesting about them?
- use as many options and arguments as possible — colours, symbols, line types, sizes, axes, titles, labels, text, legends, ...;
- explore the help pages (e.g., `?plot`, `?par`, `?points`, `?legend`, `?text`).

Send your plot to me by email (timo.adam@uni-bielefeld.de).

It's fun — but it will also help you discover the various options and arguments of the functions.



(an example — you can do better!)